Lab 2: REST API and Durable Job Queue

Kurlykov Yevgeniy | Distributed Systems | Revised 30 October 2026

1. Objective and correction

The previous implementation provided time/echo and an in-memory FIFO broker, but lacked the required job/status workflow. This revision implements POST /jobs returning HTTP 202 and job_id, followed by GET /jobs/{job_id}. A separate worker transforms message text to uppercase. The states are queued, processing and done.

2. Architecture

api.py runs a local HTTP server on 127.0.0.1:5000. broker.py implements JobStore over SQLite. consumer.py runs independently and shares the database file with the API. producer.py submits a message and polls its status. Runtime and tests use only Python standard-library modules; requirements.txt documents that no external packages are required.

3. Queue semantics

A transaction using BEGIN IMMEDIATE selects the earliest available sequence number and assigns a 30-second lease and unique token. The job remains stored during processing. Only a valid, unexpired token can acknowledge completion; result and done are committed together. A crashed worker leaves an expired lease which a restarted worker can claim again. FIFO describes claim order, not completion order with parallel workers.

4. Actual validation

Four automated tests passed in the execution environment on 30 October 2026. The HTTP integration test started a real local HTTP server, submitted a job, observed queued, invoked the worker processing function and observed done with HELLO. Additional tests verified FIFO and acknowledgement, database reopening with expired-lease recovery and stale-token rejection, and 12 unique claims across six threads. Invalid JSON/message types, unknown IDs and oversized bodies were also checked. The raw output is in test-results.txt; reproduce using python -m unittest -v.

5. Evidence boundaries

These results verify the implementation in this environment. Lease recovery was tested with a short test lease and database reopening, not by physically killing a production worker. The demonstration delay is bypassed in the HTTP test. Existing screenshots belong to the old implementation and are not evidence of the corrected workflow. A live multi-process demonstration remains part of the individual defence.

6. Assumptions and risks

API and worker must use the same database path on one host. SQLite provides persistence across ordinary process restarts, but not protection against file deletion or corruption. Work longer than 30 seconds needs lease renewal, which is outside this prototype. Repeated execution is possible; external side effects require idempotency. No authentication, TLS or distributed database is provided. Completed records are retained and the database can grow. See spec.md for the requirement-to-test mapping.

7. Reflection and individual defence

Acknowledgement after processing closes the former loss window caused by removing a message before processing. Persistence and leases trade simplicity for explicit retry semantics. The implementation does not promise exactly-once execution. The similarity concern in the assessment cannot be resolved by a code change alone: the student must explain the design, disclose assistance as required, and demonstrate the behaviour personally. DEFENSE.md lists the concepts and demonstrations to prepare.

8. Repository history

The original spec.md commit 6deeb06 precedes gitignore f299777 and implementation f50805d. The history is preserved. GIT_HISTORY.md explains how to inspect git log --reverse --stat lab2. The current specification describes the revision and is not represented as the original submission.